DESIGN AND ANALYSIS OF ALGORITHMS(LAB-5)

Name: D. Chetan Kumar

Roll no: CH.SC.U4CSE24209

1.Merge Sort

Code:

```
#include <stdio.h>

void merge(int arr[], int left, int mid, int right)
{
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    for (i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for (j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    i = 0;
    j = 0;
    k = left;

    while (i < n1 && j < n2)
```

```
    {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    while (i < n1)
        arr[k++] = L[i++];



    while (j < n2)
        arr[k++] = R[j++];
}

void mergeSort(int arr[], int left, int right)
{
    if (left < right)
    {
        int mid = (left + right) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
int main()
{
    int arr[100], n, i;
```

```
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter array elements:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    mergeSort(arr, 0, n - 1);
    printf("Sorted array:\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

Output:

```
chetan@LAPTOP-RO95NMPB:~$ nano mergeSort.c
chetan@LAPTOP-RO95NMPB:~$ gcc mergeSort.c -o mergeSort
chetan@LAPTOP-RO95NMPB:~$ ./mergeSort
Enter number of elements: 8
Enter array elements:
2 8 1 6 24 19 65 37
Sorted array:
1 2 6 8 19 24 37 65 chetan@LAPTOP-RO95NMPB:~$
```

Complexity analysis

- **Best Case:** O(n log n), When the array is already sorted or nearly sorted.
- **Average Case:** O(n log n), When the array is randomly ordered.
- **Worst Case:** O(n log n), When the array is sorted in reverse order.

2.Quick Sort

Code:

```
#include <stdio.h>


void swap(int *a, int *b)
```

```
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}
void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
```

```
        quickSort(arr, pi + 1, high);
    }
}
int main()
{
    int arr[100], n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter array elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    quickSort(arr, 0, n - 1);
    printf("Sorted array:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

Output:

```
chetan@LAPTOP-RO95NMPB:~$ nano quickSort.c
chetan@LAPTOP-RO95NMPB:~$ gcc quickSort.c -o quickSort
chetan@LAPTOP-RO95NMPB:~$ ./quickSort
Enter number of elements: 8
Enter array elements:
64
25
71
22
19
36
84
79
Sorted array:
19 22 25 36 64 71 79 84 chetan@LAPTOP-RO95NMPB:~$
```

Complexity Analysis of Bubble Sort

Best case: O(n)

Average case: O(n^2)

Worst case: O(n^2)

3.Binary Search Tree(BST)

Code:

```
#include <stdio.h>
#include <stdlib.h>

/* Structure for BST Node */
struct Node
{
    int key;
    struct Node *left, *right;
};

/* Create a new node */
struct Node *newNodeCreate(int value)
{
    struct Node *temp = (struct Node *)malloc(sizeof(struct Node));
    temp->key = value;
    temp->left = temp->right = NULL;
    return temp;
}

/* Search a node in BST */
struct Node *searchNode(struct Node *root, int target)
```

```
{
    if (root == NULL || root->key == target)
        return root;

    if (root->key < target)
        return searchNode(root->right, target);

    return searchNode(root->left, target);
}

/* Insert a node in BST */
struct Node *insertNode(struct Node *node, int value)
{
    if (node == NULL)
        return newNodeCreate(value);

    if (value < node->key)
        node->left = insertNode(node->left, value);
    else if (value > node->key)
        node->right = insertNode(node->right, value);

    return node;
}

/* Postorder traversal */
void postOrder(struct Node *root)
{
    if (root != NULL)
```

```
    {
        postOrder(root->left);
        postOrder(root->right);
        printf("%d ", root->key);
    }
}

/* Inorder traversal */
void inOrder(struct Node *root)
{
    if (root != NULL)
    {
        inOrder(root->left);
        printf("%d ", root->key);
        inOrder(root->right);
    }
}

/* Preorder traversal */
void preOrder(struct Node *root)
{
    if (root != NULL)
    {
        printf("%d ", root->key);
        preOrder(root->left);
        preOrder(root->right);
    }
}
```

```
/* Find minimum value node */
struct Node *findMin(struct Node *root)
{
    if (root == NULL)
        return NULL;
    else if (root->left != NULL)
        return findMin(root->left);
    return root;
}

/* Delete a node from BST */
struct Node *delete(struct Node *root, int x)
{
    if (root == NULL)
        return NULL;

    if (x > root->key)
        root->right = delete(root->right, x);
    else if (x < root->key)
        root->left = delete(root->left, x);
    else
    {
        /* No child */
        if (root->left == NULL && root->right == NULL)
        {
            free(root);
            return NULL;
```

```
        }
        /* One child */
        else if (root->left == NULL || root->right == NULL)
        {
            struct Node *temp;
            if (root->left == NULL)
                temp = root->right;
            else
                temp = root->left;

            free(root);
            return temp;
        }
        /* Two children */
        else
        {
            struct Node *temp = findMin(root->right);
            root->key = temp->key;
            root->right = delete(root->right, temp->key);
        }
    }
    return root;
}

/* Main function */
int main()
{
    struct Node *root = NULL;
```

```
    root = insertNode(root, 50);
    insertNode(root, 30);
    insertNode(root, 20);
    insertNode(root, 40);
    insertNode(root, 70);
    insertNode(root, 60);
    insertNode(root, 80);

    if (searchNode(root, 60) != NULL)
        printf("60 found\n");
    else
        printf("60 not found\n");

    printf("Postorder: ");
    postOrder(root);
    printf("\n");

    printf("Preorder: ");
    preOrder(root);
    printf("\n");

    printf("Inorder: ");
    inOrder(root);
    printf("\n");

    return 0;
}
```

Output:

```
chetan@LAPTOP-RO95NMPB:~$ nano binarysearchtree.c
chetan@LAPTOP-RO95NMPB:~$ gcc binarysearchtree.c -o binarysearchtree
chetan@LAPTOP-RO95NMPB:~$ ./binarysearchtree
60 found
Postorder: 20 40 30 60 80 70 50
Preorder: 50 30 20 40 70 60 80
Inorder: 20 30 40 50 60 70 80
```

- All nodes in the left subtree of a node contain values **strictly less** than the node's value.
- All nodes in the right subtree of a node contain values **strictly greater** than the node's value.